NNDL project report template

Edoardo Furlan†, Andrea Marigo†, Francesca Zorzi†

***Abstract*—I suggest to write the Abstract as the very last thing. You may sketch it at the beginning, but then always finalize it at the end.**

***Index Terms*—Neural Networks, Convolutional Neural Networks, Generative Adversarial Networks, Symbolic-Domain Music**

I. Introduction

The task of generating music is a frontier in Artificial Intelligence, since it poses the challenge of training a network to be creative, but with the rigid constraints imposed by music theory. While some architectures are already capable of generating technically correct sequences, capturing the key points of realistic, human-like compsitions, remains an interesting topic. This is a praticularly interesting problem if seen in the scenario of creating assistive tools for composers: both experienced and non experienced ones would benefit by these tools in order to improve creativity and exploration of genres. What we propose is an architecture that aims no only to model the placement of notes, but also the evolution of the themaic they represent.

In music generation, coherence is the main objective. That's why a simple RNN architecture seems to be the most tappropriate solution, as it's structure aims at learning sequences in order to predict the next values. However, MidiNet [1] showed that convolutional GANs can overcome the issue. The main problem is that in this particular configuration, the discriminator tends to be better than the generator, eventually quickly reaching a stable 100% accuracy. The network that we propose tries to enhance the capabilities of MidiNet by adding to the generation process a set of residual layers unit that should help in learning long-term relations. Additionally, as mentioned in MidiNet paper, to avoid the vanishing gradient problem, we changed the loss function from a conventional cross-entropy to a Wasserstein loss, and in particular we chose the version that uses gradient penalty [2], [3]. In this scenario the Discriminator is considered as a Critic, which assigns a score (not a probability) to the generated music. Finally, we also modified the critic network with an improved set of convolutional layers, in order to improve its capability of extracting the relevant features from the music. These changes aim to improve the quality of the game played by Generator and Discriminator, forcing more balanced capabilities to both the structures, which should eventually lead to a more realistic result.

Summary of contributions:

†Department of Information Engineering, University of Padova, email: {edoardo.furlan.3, andrea.marigo.3, francesca.zorzi.3}@studenti.unipd.it

- WGAN-GP (Wasserstein GAN - Gradient Penalty): loss is modified to help longer and more stable training
- Residual Units: The generator is enriched with residual units to provide more direct flow of the gradient.
- Critic structure: the structure of the critic is improved to enable more effective feature extraction.
- Da aggiungere

In the paper we follow this structure. In Section II we describe the base model we started from, in section III we explain the general processing pipeline. In Section IV we describe the dataset we used and the preprocessing techniques we adopted. In Section V we explain our model and we conclude in Section VI showing the results we obtained.

II. Related Work

Our network is based on the MidiNet architecture [1]. What this architecture introduced is the idea of conditioning the generation process by combining the outputs of convolutional layer of the Conditioner network to the input of the layers of the Generator. In this way the network is able to produce music coherent with previous bars. However, while training an exact copy of the network we noted the typical struggles of GANs: the Discriminator was quickly able to distinguish real from fake music, leading to a completely random Generation process. Our work then focused on finding ways to improve the balance between the two networks. The main idea was the modification of the loss function. As already said, the MidiNet paper opens to the possibiliy of changing to a loss that avoids vanishing gradients, which is the one we chose in our network

III. Processing Pipeline

A schematic represtation of the network is shown in Figure **Inserire figura**

A. Generator Res-CNN and Critic CNN

We used a DCGAN architecture which has the objective of training a Critic CNN to distinguish between real samples and samples generated by a Generator Res-CNN, which aims at fooling the Critic. For the generation process we used random noises $z \in \mathbb{R}^{100}$ sampled from a Normal Gaussian distribution, which are passed through a series of transposed conolutional layers and residual blocks to produce the correct output matrix representing the music bar. The Critic CNN is a convolutional network which takes as input a music bar and outputs a single scalar value representing the "score" of the bar, i.e. how likely it is to be a real music bar.

This produces a minimax game between Generator and Critic, which ensures non-saturating gradients for both networks, due to the linearity of the Critic's output. More details about the loss structure will be explained in Section V-B.

In order to deal with mode collapse, [1] introduced feature matching and one-sided label smoothing. After trying these ideas in our network, wefound that they were not enough to solve the problem. As a consequence, to obtain more control over the game we also introduced a minibatch discrimination block in the Critic CNN. This block computes the standard deviation of each feature across the batch, averages it to a single scalar $s = \text{mean}_{c,h,w}(\text{std}b(Ib, c, h, w))$ and concatenates it as an additional channel. The idea is to provide the Critic with a statistical point of view of the input samples, forcing the Generator to produce music without collapsing to one single mode.

B. Conditioner Network

Our network also takes the contidioning idea of [1]. The Conditioner Network is a CNN which takes as input a conditioning matrix and concatenates the output of each convolutional layer to the output of the corresponding layer of the generator, leading to an influenced generation process.

IV. Signals & Features

To generate coherent symbolic-domain music, our system relies on a pipeline that processes MIDI files into a highly structured matrix representation, suitable for training a Deep Convolutional Generative Adversarial Network (DCGAN). At a high level, the pipeline consists of data ingestion, feature extraction (chords and monophonic melodies), formatting, and adversarial training. While the generative architecture and the learning mechanism will be detailed in Section V, this section focuses on the dataset selection and the preprocessing methodologies required to map the MIDI data into effective neural network inputs.

A. Dataset Selection

The dataset chosen to train our model is the Maestro V3.0.0 [4]. This collection provides over 200 hours of virtuosic piano performances, already processed and formatted as MIDI files. The primary motivation behind this choice is consistency: the dataset exclusively features piano performances, and the tempo is standardized (120 bpm). This temporal uniformity is a crucial property, as it allowed us to parse all MIDI files using a constant sampling frequency. Consequently, we obtained a homogeneous data distribution that reduces the variance unrelated to the actual musical composition, thereby helping the model to better capture and reproduce the underlying melodic distribution.

To evaluate the Generative Adversarial Network (GAN) rigorously and prevent data leakage, we adopted a hybrid dataset splitting approach built upon this homogeneous corpus:

- **Test Set:** Prior to any preprocessing or segmentation, a subset of three complete MIDI files from the Maestro dataset was strictly isolated. These completely unseen tracks are reserved exclusively for the final Conditioned Autoregressive Generation evaluation. Crucially, they provide the priming bar and the ground-truth harmonic progression required to properly constrain the generation process.
- **Training and Validation Sets:** The remainder of the musical corpus was processed into fixed-length matrices and randomly partitioned into a Training Set (90%) and a Validation Set (10%). The training set is utilized for weight optimization over **[number of epochs]** epochs. Concurrently, the validation set serves to monitor training stability over time.

Although MIDI files are structured, they are inherently event-based and highly polyphonic, making them unsuitable for direct ingestion into standard convolutional layers. Following the guidelines of related works in symbolic music generation [1], we mapped the data into a 2-D matrix (piano roll) representation.

Assuming a standard 4/4 time signature, we defined a spatial resolution of 16 time steps per bar and a pitch range of 128 discrete MIDI notes. The entire dataset was sequentially segmented into independent windows of 8 consecutive bars. Within our framework, two bars are considered temporally consecutive only if they belong to the same 8-bar segment. To enable the Generator to learn the onset of musical phrases without prior conditioning, the first bar of every segment is intentionally treated as the subsequent bar of an empty (silence) matrix.

B. Feature Extraction and Monophonic Conversion

Since our target generative architecture is designed primarily for monophonic melodies conditioned on harmonic contexts, we applied two distinct feature extraction steps to the segments:

1) **Chord Extraction:** The Maestro dataset does not provide explicit chord labels. To extract the underlying harmonic structure, we computed Chroma features for each bar, aggregating the pitch energy across the 12 pitch classes. By comparing the resulting Chroma vectors against predefined templates through cross-correlation, we assigned an integer index (from 0 to 24) to each bar. This index efficiently encodes 12 major chords, 12 minor chords, and a silence/no-chord state.
2) **Monophonic Conversion and Prolonging:** The original piano rolls feature complex polyphony and sparse note activations. To extract the main melody, we applied a highest-note priority algorithm: for every time step, only the note with the highest pitch and velocity was retained, explicitly silencing the rest. Furthermore, to prevent the network from generating fragmented, staccato-like outputs, we employed a forward-fill strategy to bridge brief silences. If a time step was empty, the activation of the previous note was prolonged. This artificial continuity enforces legato phrasing, making it easier for the convolutional kernels to learn stable temporal dependencies.
3) **Binarization and Pitch Clipping:** To focus the learning process on the most musically expressive register and reduce data sparsity, each extracted segment was binarized

(setting all non-zero velocities to 1) and pitch-clipped, zeroing out any note outside the $[24, 96]$ range.

Overall, this extraction and filtering procedure yielded a total of 44,123 valid segments, each formatted as a 128×16 matrix per bar.

V. Learning Framework

A. Model Architecture

B. Wasserstein Loss and Gradient Penalty

We start from the output of the discriminator $D(x)$. The forward pass does not compute the final sigmoid activation, which shrinks the output values to the range $[0, 1]$ to assign the probability.

To compute generator and discriminator losses, we use the following formulas:

$$L_D = \mathbb{E}_{\tilde{x}\sim P_g}[D(\tilde{x})] - \mathbb{E}_{x\sim P_r}[D(x)] \tag{1}$$

$$L_G = -\mathbb{E}_{\tilde{x}\sim P_g}[D(\tilde{x})] \tag{2}$$

Where we can see that the both losses become linear with respect to the critic output, avoiding null gradients enforced by a Sigmoid.

Regarding the Gradient Penalty, it is introduced to enforce the Lipschitz constraint on the discriminator, which acts as a regularization term on the gradient norm. The total loss of the discriminatore is then computed as:

$$\begin{aligned} L = &\underbrace{\mathbb{E}_{\tilde{x}\sim P_g}[D(\tilde{x})] - \mathbb{E}_{x\sim P_r}[D(x)]}_{\text{Wasserstein Loss}} \\ &+ \underbrace{\lambda \mathbb{E}_{\hat{x}\sim P_{\hat{x}}}\left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2\right]}_{\text{Gradient Penalty}} \end{aligned} \tag{3}$$

Note that $\hat{x}$ is a sampled taken from a random interpolation between real and fake samples. It can be seen as a random point along the line connecting the two distributions. This forces the Discriminator to keep small gradients along the trajectories connecting real and fake samples.

C. Hyperparameter tuning

The training was influenced mainly by three hyperparameters:

1) **Gradient penalty coefficient (λ):** This parameter was never modified and was always set to 10.0 as in the original paper.
2) **Feature matching weight (w_{fm}):** This parameter has the same function as the one presented in MidiNet. However, during the last trainings we set it to a very low value since we noted it could cause mode collapse, as it allows the generator to give less importance to the adversarial loss.
3) **Number of critic steps per generator step (n_c):** This parameter was fundamental after the change of loss function and critic structure, as it allowed us to balance the training of the two networks. We noticed that a value between 3 and 5 was optimal for the training, to allow the discriminator to learn enough to provide useful gradients to the generator, which otherwise tended to collapse

VI. Results

VII. Concluding Remarks

References

[1] L.-C. Yang, S.-Y. Chou, and Y.-H. Yang, "MidiNet: A Convolutional Generative Adversarial Network for Symbolic-Domain Music Generation," in *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*, (Suzhou, China), Oct. 2017.

[2] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of wasserstein GANs," in *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS)*, (Long Beach, CA, USA), Dec. 2017.

[3] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, (Sydney, Australia), Aug. 2017.

[4] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the MAESTRO dataset," in *International Conference on Learning Representations*, 2019.

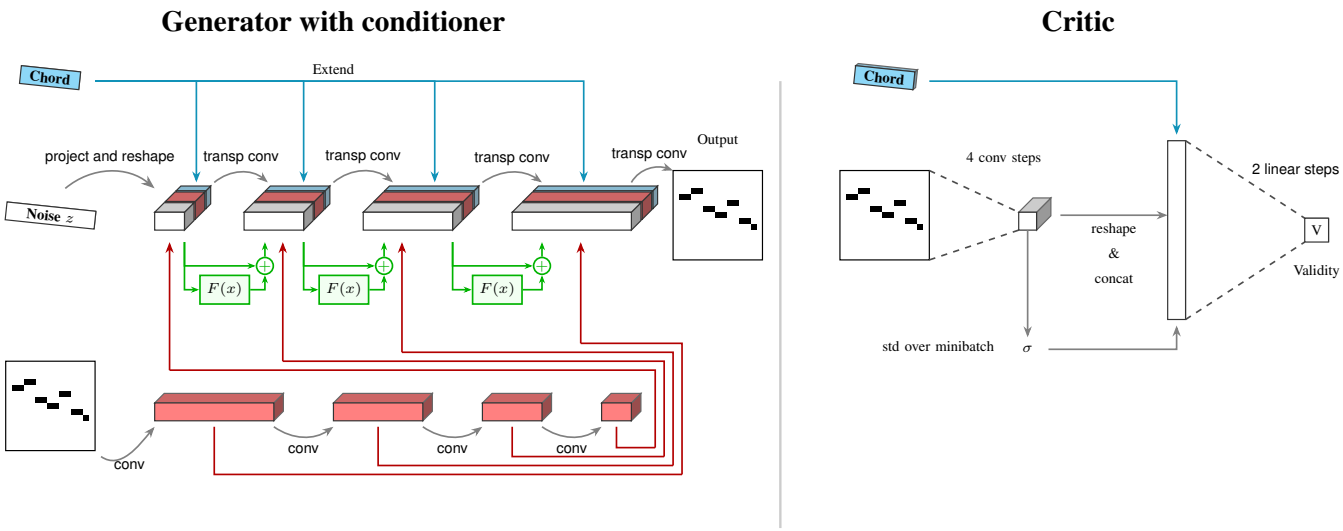

Fig. 1: Model architecture: Generator with conditioner (left) and Discriminator CNN (right).